

Terraform-02

- 1). Watch the **Terraform-02** video.
- 2). Execute all the **templates** shown in the video.

```
main.tf > resource "random_pet" "mypet" > abc separator
1  resource "local_file" "pet" {
2    filename = "pets.txt"
3    content = "we love pets!"
4  }
5  resource "random_pet" "mypet" {
6    prefix = "MR"
7    separator = "."
8    length = "1"
9  }
10
```

Now lets perform terraform init

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/local...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)
- Installing hashicorp/random v3.9.0...
- Installed hashicorp/random v3.9.0 (signed by HashiCorp)

Initializing the backend...
```

Terraform plan

```
PS C:\Users\DELL\Desktop\Terraform> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# local_file.pet will be created
+ resource "local_file" "pet" {
  + content          = "we love pets!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1       = (known after apply)
  + content_sha256     = (known after apply)
  + content_sha512     = (known after apply)
  + directory_permission = "0777"
  + file_permission    = "0777"
  + filename          = "pets.txt"
  + id                = (known after apply)
}

# random_pet.mypet will be created
+ id = (known after apply)
```

Terraform apply

```
PS C:\Users\DELL\Desktop\Terraform> terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

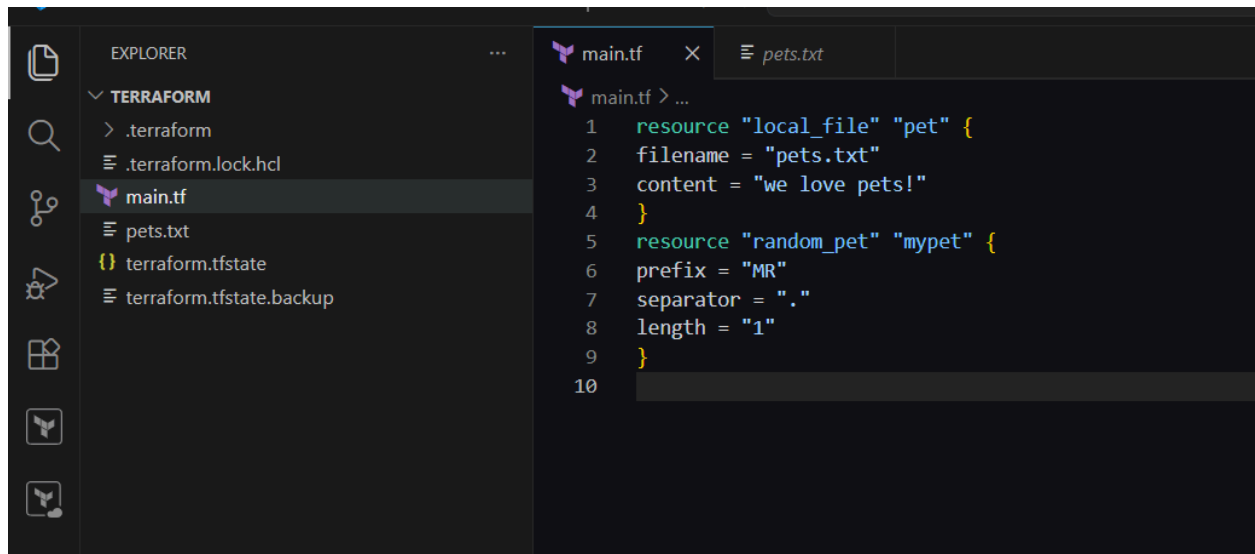
- + create

Terraform will perform the following actions:

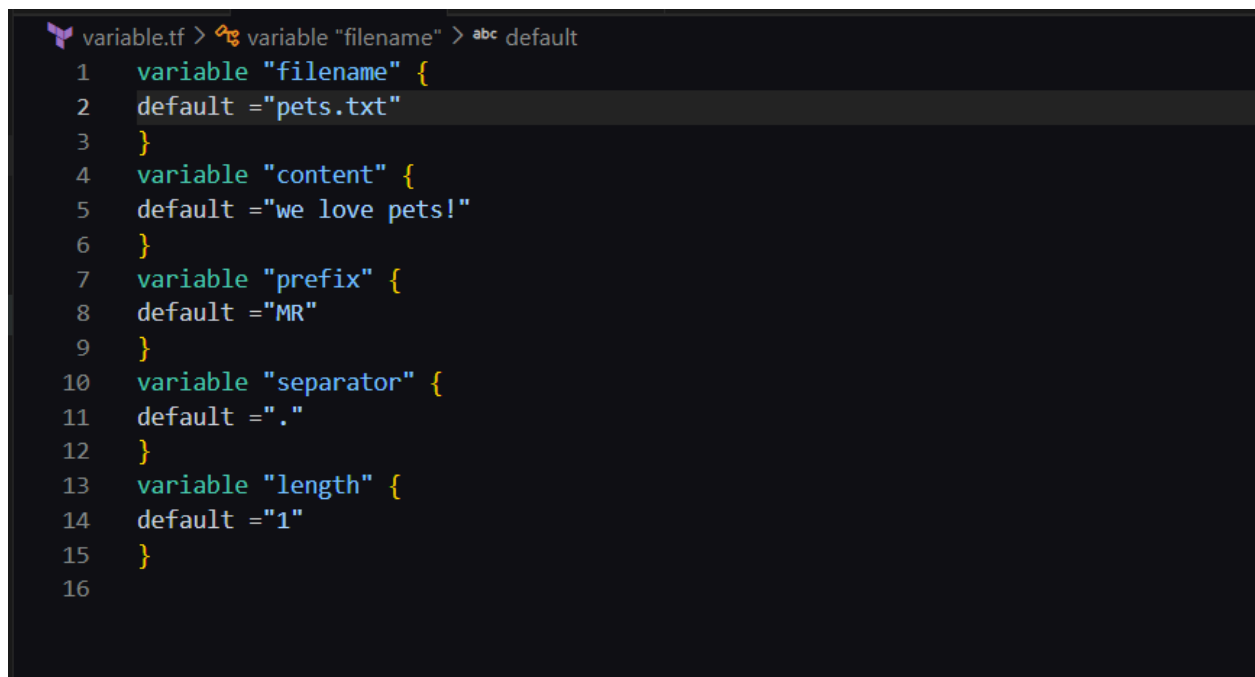
```
# local_file.pet will be created
+ resource "local_file" "pet" {
  + content          = "we love pets!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1       = (known after apply)
  + content_sha256     = (known after apply)
  + content_sha512     = (known after apply)
  + directory_permission = "0777"
  + file_permission    = "0777"
  + filename          = "pets.txt"
  + id                = (known after apply)
}

# random_pet.mypet will be created
+ resource "random_pet" "mypet" {
```

ln 7:6

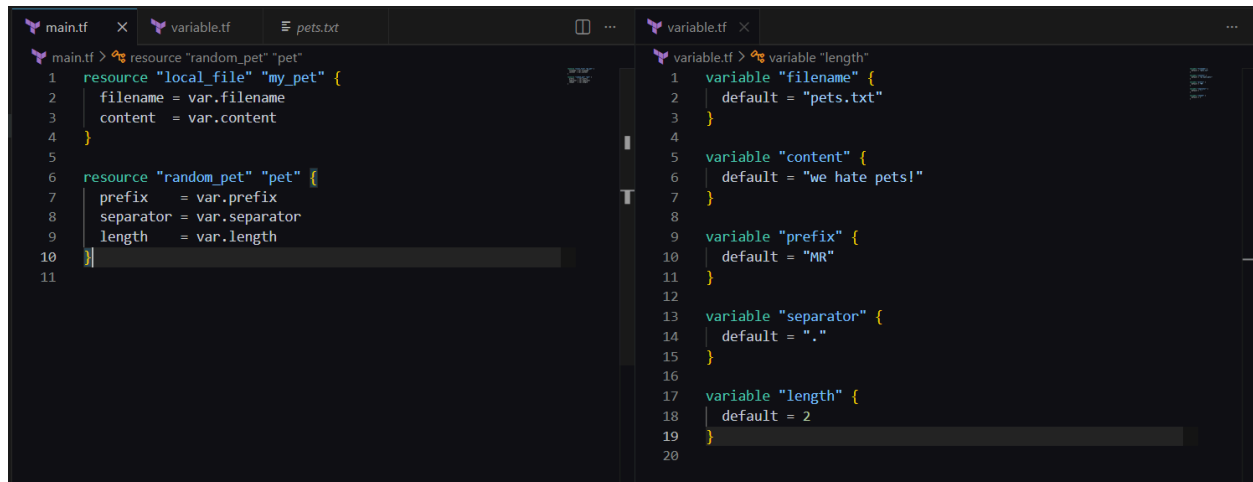


Now lets create variable.tf file



Instead of hard coding the values inside main.tf file we have created a variables.tf. So

main.tf will use those variables with var.content, var.filename etc

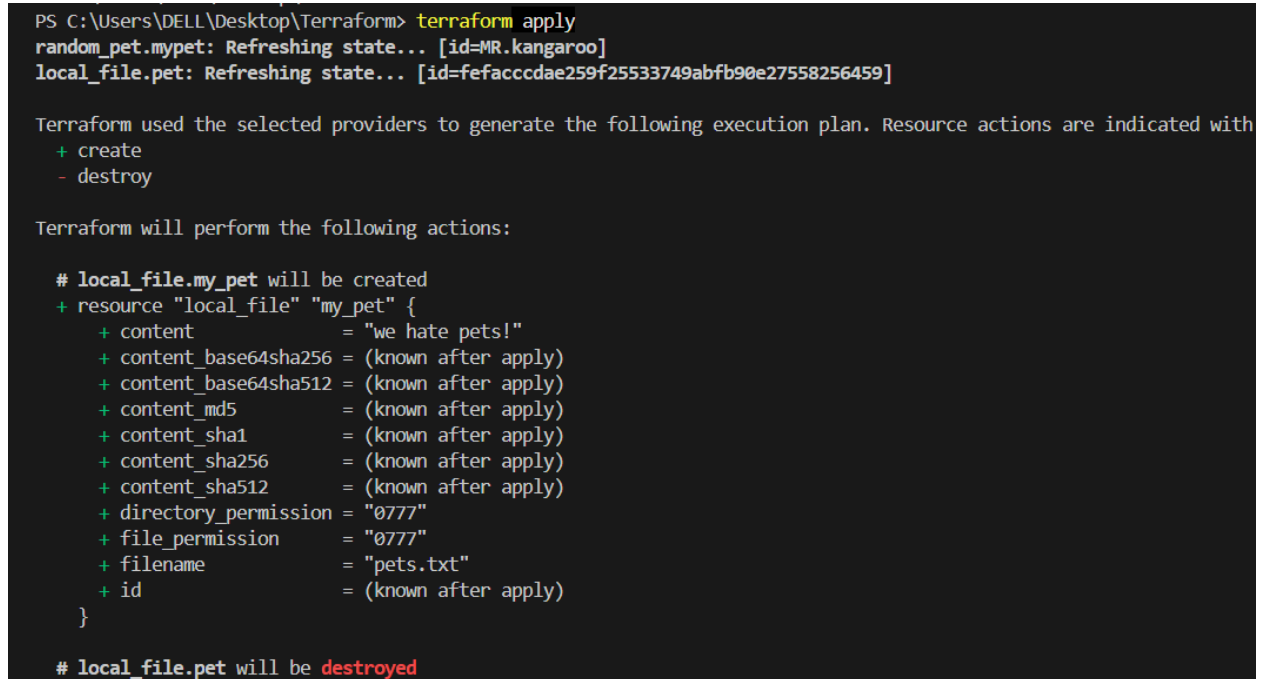


The screenshot shows a code editor with two files open. The left pane shows `main.tf` with two resource blocks. The first block creates a `local_file` resource named `my_pet` with attributes `filename` and `content` set to `var.filename` and `var.content` respectively. The second block creates a `random_pet` resource named `pet` with attributes `prefix`, `separator`, and `length` set to `var.prefix`, `var.separator`, and `var.length` respectively. The right pane shows `variable.tf` with five variable definitions: `filename` (default `"pets.txt"`), `content` (default `"we hate pets!"`), `prefix` (default `"MR"`), `separator` (default `"."`), and `length` (default `2`).

```
main.tf
1 resource "local_file" "my_pet" {
2   filename = var.filename
3   content  = var.content
4 }
5
6 resource "random_pet" "pet" {
7   prefix    = var.prefix
8   separator = var.separator
9   length    = var.length
10 }
11

variable.tf
1 variable "filename" {
2   default = "pets.txt"
3 }
4
5 variable "content" {
6   default = "we hate pets!"
7 }
8
9 variable "prefix" {
10  default = "MR"
11 }
12
13 variable "separator" {
14   default = "."
15 }
16
17 variable "length" {
18   default = 2
19 }
20
```

Now lets perform terraform apply



The screenshot shows a terminal window with the output of a `terraform apply` command. The command is executed from the directory `C:\Users\DELL\Desktop\Terraform`. The output shows that the state is being refreshed for `random_pet.mypet` and `local_file.pet`. The execution plan indicates that `local_file.my_pet` will be created and `local_file.pet` will be destroyed. The plan for `local_file.my_pet` shows the following attributes: `content` (value `"we hate pets!"`), `content_base64sha256`, `content_base64sha512`, `content_md5`, `content_sha1`, `content_sha256`, `content_sha512`, `directory_permission` (value `"0777"`), `file_permission` (value `"0777"`), `filename` (value `"pets.txt"`), and `id`.

```
PS C:\Users\DELL\Desktop\Terraform> terraform apply
random_pet.mypet: Refreshing state... [id=MR.kangaroo]
local_file.pet: Refreshing state... [id=fefacccdae259f25533749abfb90e27558256459]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
+ create
- destroy

Terraform will perform the following actions:

# local_file.my_pet will be created
+ resource "local_file" "my_pet" {
+   content                = "we hate pets!"
+   content_base64sha256   = (known after apply)
+   content_base64sha512   = (known after apply)
+   content_md5            = (known after apply)
+   content_sha1           = (known after apply)
+   content_sha256         = (known after apply)
+   content_sha512         = (known after apply)
+   directory_permission    = "0777"
+   file_permission        = "0777"
+   filename               = "pets.txt"
+   id                    = (known after apply)
}

# local_file.pet will be destroyed
```

```
# random_pet.mypet will be destroyed
# (because random_pet.mypet is not in configuration)
- resource "random_pet" "mypet" {
  - id          = "MR.kangaroo" -> null
  - length      = 1 -> null
  - prefix      = "MR" -> null
  - separator   = "." -> null
}

# random_pet.pet will be created
+ resource "random_pet" "pet" {
  + id          = (known after apply)
  + length      = 2
  + prefix      = "MR"
  + separator   = "."
}
```

Plan: 2 to add, 0 to change, 2 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

```
# random_pet.pet will be created
+ resource "random_pet" "pet" {
  + id          = (known after apply)
  + length      = 2
  + prefix      = "MR"
  + separator   = "."
}

Plan: 2 to add, 0 to change, 2 to destroy.

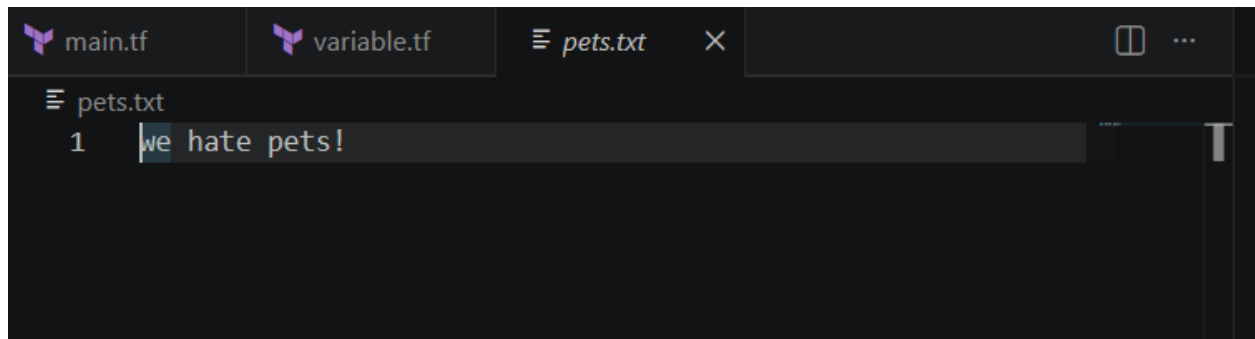
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.pet: Destroying... [id=fefacccdae259f25533749abfb90e27558256459]
random_pet.mypet: Destroying... [id=MR.kangaroo]
local_file.pet: Destruction complete after 0s
random_pet.pet: Creating...
local_file.my_pet: Creating...
random_pet.mypet: Destruction complete after 0s
random_pet.pet: Creation complete after 0s [id=MR.measured.clam]
local_file.my_pet: Creation complete after 0s [id=d1f48b051f876761cdd2ab566ddc899593fafa85]

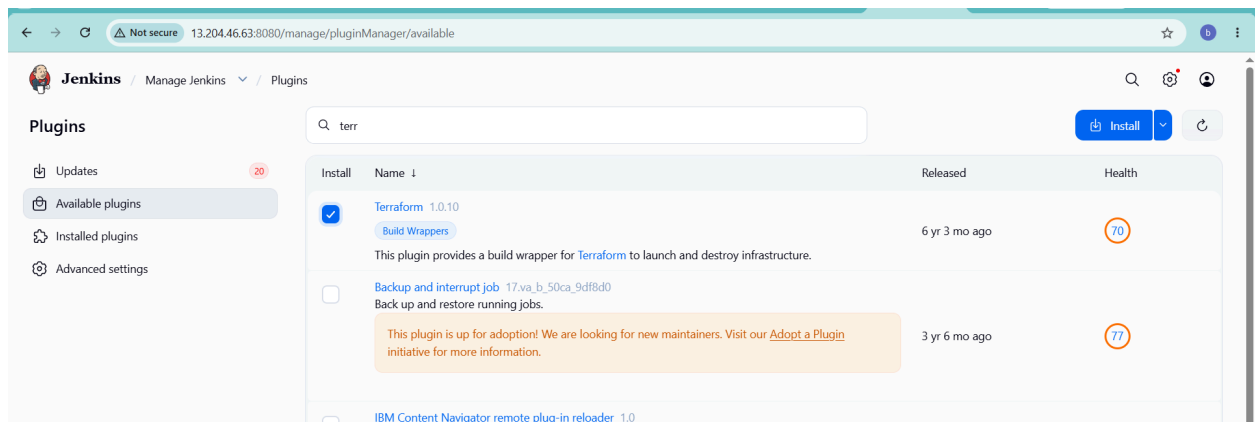
Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
PS C:\Users\DELL\Desktop\Terraform> []
```

Here content is replaced



3). Integrate **Terraform** in **Jenkins** using the **Terraform** plugin.

Lets install terraform plugin in Jenkins



Add tools:

Gave name & terraform path

Terraform installations

+ Add Terraform

Terraform

Name

Terraform

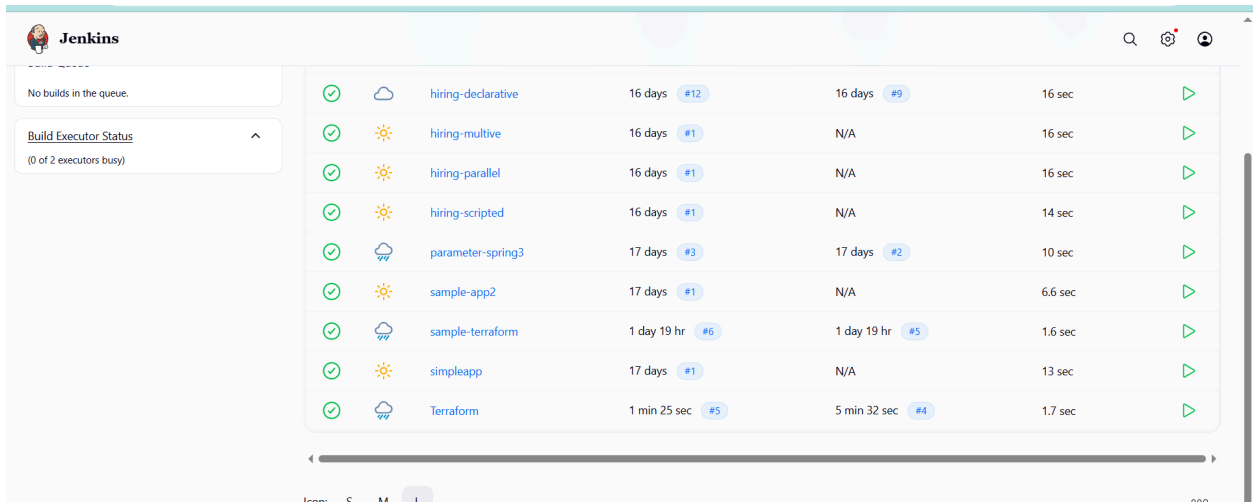
Install directory

/usr/bin

☐ Install automatically ?

+ Add Terraform

Create a new item



The screenshot shows the Jenkins dashboard. On the left, there's a sidebar with a 'Build Executor Status' section indicating '(0 of 2 executors busy)'. The main area displays a table of recent builds. Each row includes a status icon (green checkmark), a cloud icon, the build name, the age, the build number, the duration, and a play button icon.

Status	Icon	Build Name	Age	Build #	Duration	Action	
✓	☁	hiring-declarative	16 days	#12	16 days #9	16 sec	▶
✓	☀	hiring-multive	16 days	#1	N/A	16 sec	▶
✓	☀	hiring-parallel	16 days	#1	N/A	16 sec	▶
✓	☀	hiring-scripted	16 days	#1	N/A	14 sec	▶
✓	☁	parameter-spring3	17 days	#3	17 days #2	10 sec	▶
✓	☀	sample-app2	17 days	#1	N/A	6.6 sec	▶
✓	☁	sample-terraform	1 day 19 hr	#6	1 day 19 hr #5	1.6 sec	▶
✓	☀	simpleapp	17 days	#1	N/A	13 sec	▶
✓	☁	Terraform	1 min 25 sec	#5	5 min 32 sec #4	1.7 sec	▶

Lets create a main.tf file

```
[root@ip-172-31-14-229 Terraform]# cd
[root@ip-172-31-14-229 ~]# cd /var/lib/jenkins/workspace/Terraform
[root@ip-172-31-14-229 Terraform]#
```

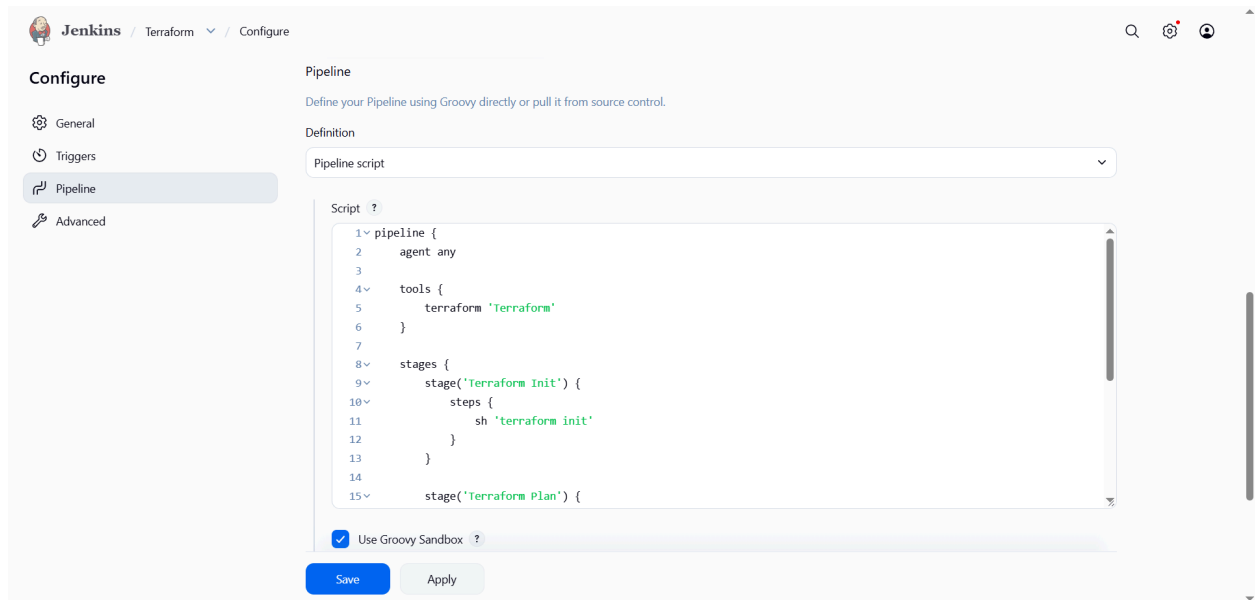
i-0532703582a1480a1 (Jenkins)

PublicIPs: 13.204.46.63 PrivateIPs: 172.31.14.229

```
[root@ip-172-31-14-229 Terraform]# cd
[root@ip-172-31-14-229 ~]# cd /var/lib/jenkins/workspace/Terraform
[root@ip-172-31-14-229 Terraform]# cat main.tf
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "demo" {
  filename = "terraform-from-jenkins.txt"
  content = "Terraform executed successfully from Jenkins using Terraform Plugin"
}
[root@ip-172-31-14-229 Terraform]#
```

Now lets create a pipeline job:



```
pipeline {  
  agent any  
  
  tools {  
    terraform 'Terraform'  
  }  
  
  stages {  
    stage('Terraform Init') {  
      steps {  
        sh 'terraform init'  
      }  
    }  
  
    stage('Terraform Plan') {  
      steps {  
        sh 'terraform plan'  
      }  
    }  
  }  
}
```



```

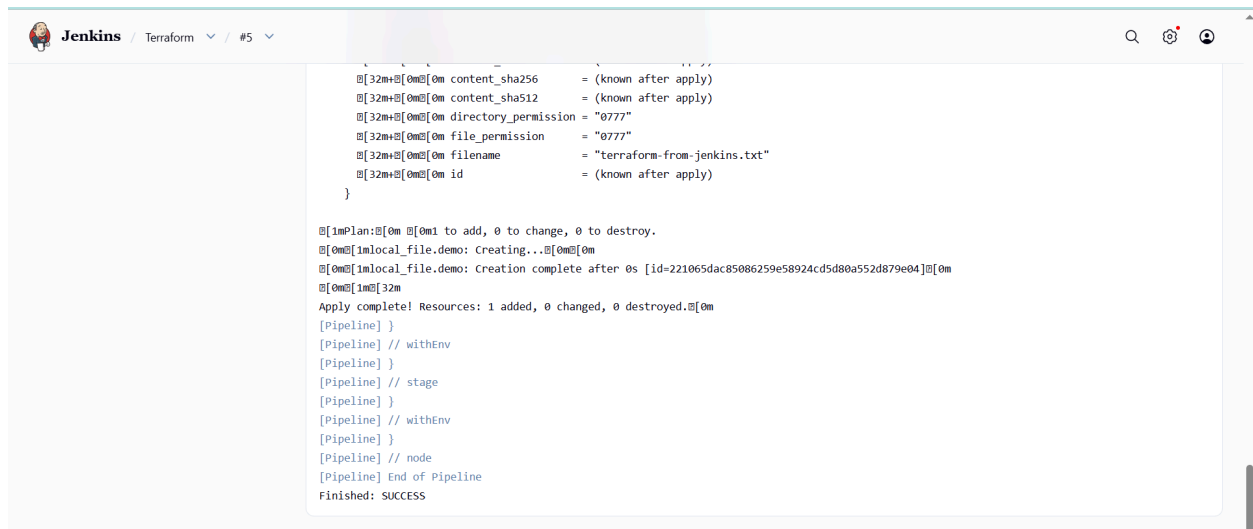
stage('Terraform Apply') {
  steps {
    sh 'terraform apply -auto-approve'
  }
}
}
}

```

Save & build

Build successfully

Verify console output:



The screenshot shows the Jenkins web interface with the console output of a Terraform apply command. The output is as follows:

```

[32m+[0m[0m content_sha256      = (known after apply)
[32m+[0m[0m content_sha512        = (known after apply)
[32m+[0m[0m directory_permission   = "0777"
[32m+[0m[0m file_permission        = "0777"
[32m+[0m[0m filename                = "terraform-from-jenkins.txt"
[32m+[0m[0m id                      = (known after apply)
}

[1mPlan:[0m [0m1 to add, 0 to change, 0 to destroy.
[0m[1mlocal_file.demo: Creating...[0m[0m
[0m[1mlocal_file.demo: Creation complete after 0s [id=221065dac85086259e58924cd5d80a552d879e04][0m
[0m[1m[32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```